

CS-317 Operating Systems



Lecture 10



Threads

Book 1 Chapter 4

In the last lecture...



- ▶ We explored kernel level threads.
- ▶ We explored the combined approach to threads.
- ▶ We talked in detail about symmetric multiprocessors.
- ▶ We realized that SMPs and multithreading strengthen each other.
- ▶ We discussed the design issues of an operating system for SMPs.

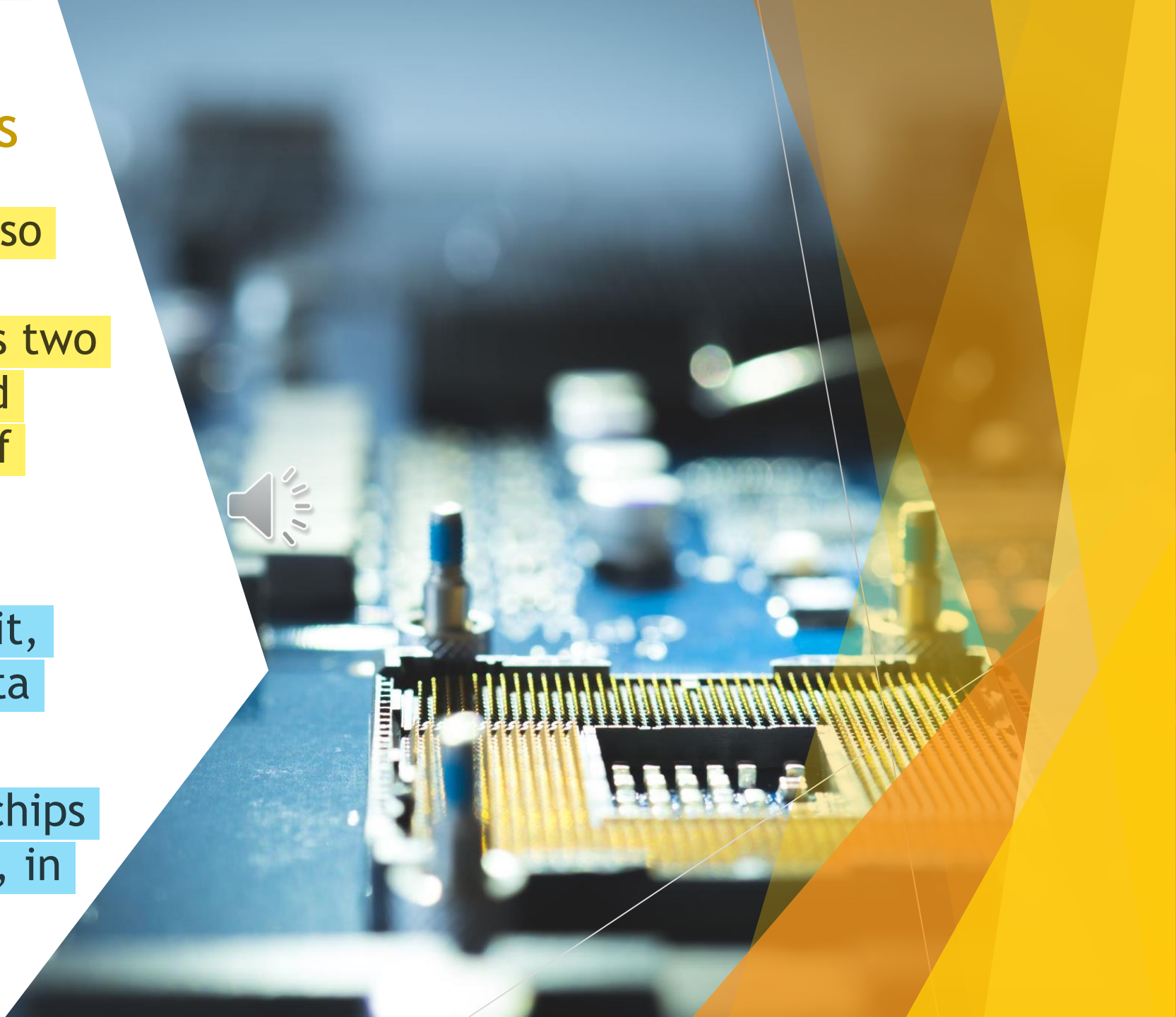


Multithreading, Multiprocessors and Multicore Systems

Book 1 Chapters 1, 2 and 4
Sections 1.8, 2.6 and 4.3

Multicore computers

- ▶ A multicore computer, also known as a **chip multiprocessor**, combines two or more processors (called **cores**) on a single piece of silicon (called a **die**).
- ▶ Each core consists of registers, ALU, control unit, and L1 instruction and data caches.
- ▶ Contemporary multicore chips also include L2 cache and, in some cases, L3 cache.



Motivation for the development of multicore computers

- ▶ For decades, microprocessor systems have experienced a steady increase in performance due to:
 - ▶ Increase in clock frequency,
 - ▶ Moving cache memory closer to the processor,
 - ▶ Increased exploitation of parallelism in instruction execution and memory access.
- ▶ Now, designers have come up against practical limits in the ability to achieve greater performance by means of more complex processors.
- ▶ Designers have found that the best way to improve performance is to put multiple processors and a substantial amount of cache memory on a single chip.

Intel Core i7

- ▶ Intel Core i7 includes four x86 processors.
- ▶ Intel uses prefetching, in which the hardware attempts to fill the caches speculatively with data that's likely to be requested soon.

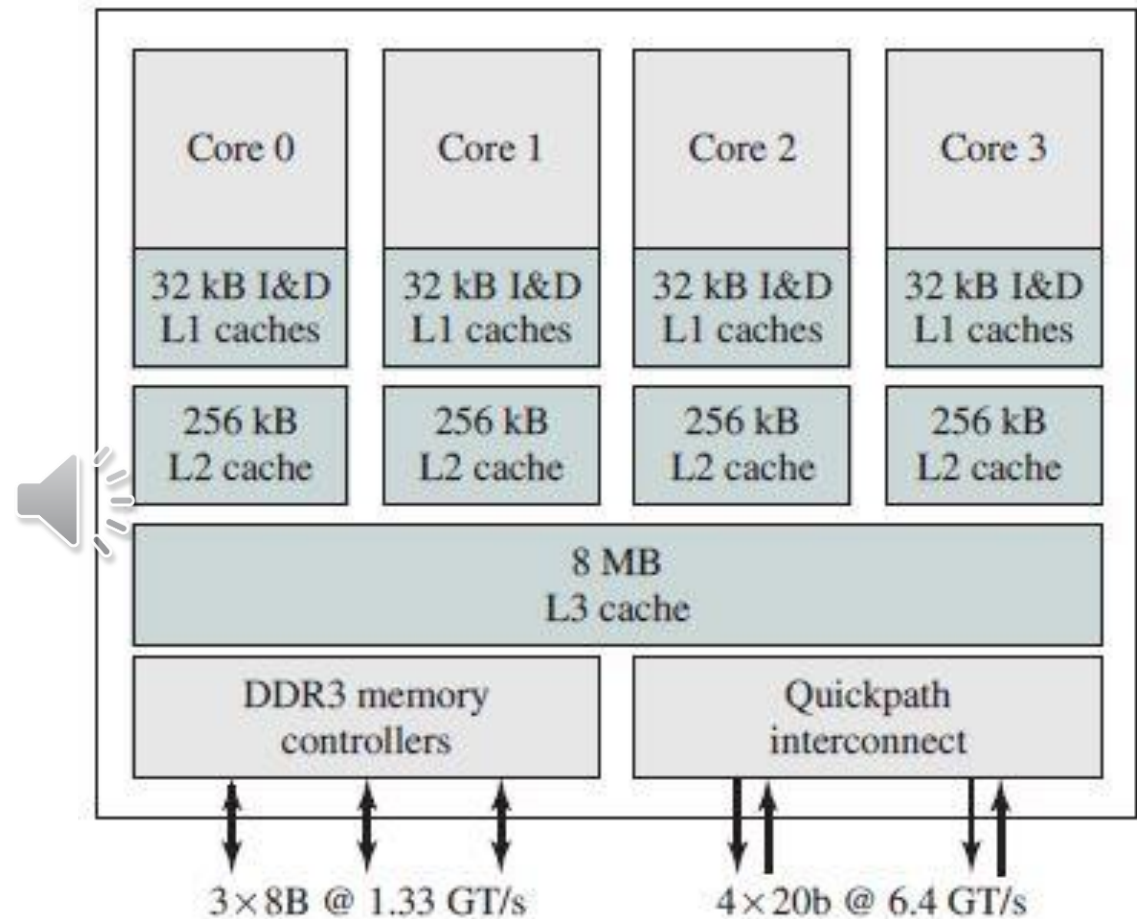


Figure 1.20 Intel Core i7 Block Diagram

Intel Core i7



- ▶ Intel Core i7 includes four x86 processors.
- ▶ Intel uses prefetching, in which the hardware attempts to fill the caches speculatively with data that's likely to be requested soon.
- ▶ The on-chip DDR3 memory controller supports three channels that are 8 bytes wide for a total bus width of 192 bits, for an aggregate data rate of up to 32 GB/s.

Memory Bandwidth = number of bytes read out of or written into a memory module per unit time.

T stands for transfer.

$$\begin{aligned}\text{Number of bytes} &= 8 \times 3 \\ &= 24\end{aligned}$$

$$\begin{aligned}\text{Number of bits} &= 24 \times 8 \\ &= 192\end{aligned}$$

$$\text{Number of transfers} = 1.33 \text{ GT/s}$$

$$\begin{aligned}\text{Memory bandwidth} &= 1.33 \times 24 \\ &= 31.92 \text{ GB/s}\end{aligned}$$

mem bandwidth = number of transefrs * channels*channel size *

Intel Core i7

- ▶ The QuickPath Interconnect (QPI) enables high-speed communications among connected processor chips.
- ▶ The QPI link operates at 6.4 GT/s (transfers per second).

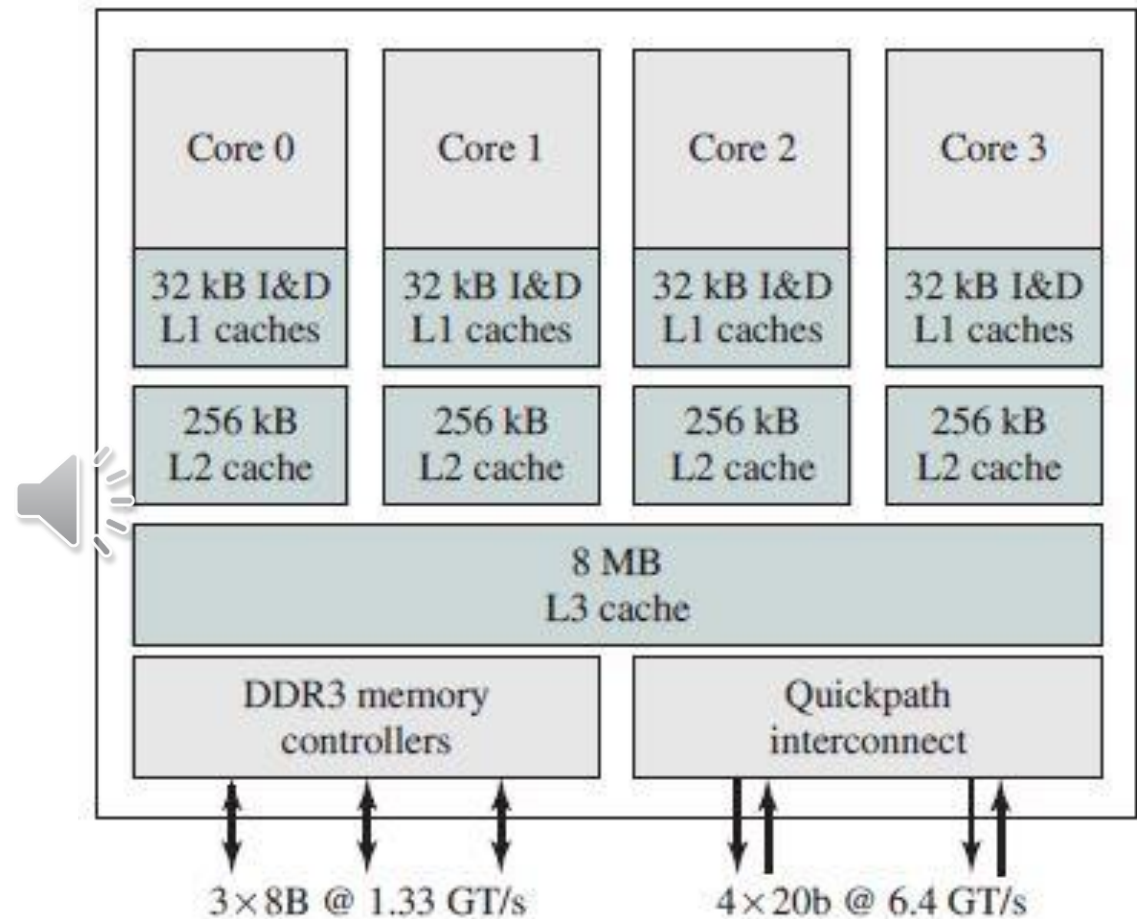


Figure 1.20 Intel Core i7 Block Diagram

Intel Core i7

- ▶ The QuickPath Interconnect (QPI) enables high-speed communications among connected processor chips.
- ▶ The QPI link operates at 6.4 GT/s (transfers per second).
- ▶ At 16 bits per transfer, that adds up to 12.8 GB/s; and since QPI links involve dedicated bidirectional pairs, the total bandwidth is 25.6 GB/s.

Interconnection Bandwidth = number of bytes read out or written into an interconnection device per unit time.

T stands for transfer.

 Nature of QPI links: Bidirectional

Operation rate of QPI = 6.4 GT/s

Bits per transfer = 16 bits/T

= 2 bytes/T

Interconnection Bandwidth

= $6.4 \times 2 \times 2$

= 25.6 GB/s

GT = giga transfer = 10^9

Multicore OS Considerations



Design of an operating system for multicores

Design concerns

- ▶ The considerations for multicores include all design issues for SMPs.
- ▶ With each succeeding processor technology generation, the number of cores and the amount of shared and dedicated cache memory increases, so that we are now entering the era of "many-core" systems.
- ▶ The design challenges for a many-core multicore system are:
 - ▶ To efficiently harness the multicore processing power, and
 - ▶ Intelligently manage the on-chip resources.



Potential for parallelism

- ▶ The potential for parallelism exists at three levels in a contemporary multicore system:



Potential for parallelism

- ▶ The potential for parallelism exists at three levels in a contemporary multicore system:

- ▶ Hardware parallelism within each core processor, known as instruction level parallelism,

1	$e = a + b$
2	$f = c + d$
3	$m = e * f$

- ❑ ILP is about parallel execution of a sequence of instructions belonging to a specific thread of a process.
 - ❑ Operation 3 depends on the results of operations 1 and 2.
 - ❑ Operations 1 and 2 do not depend on any other operation.
 - ❑ If each operation completes in one unit of time, these instructions can be completed in two units of time, giving an ILP of 3/2.

Potential for parallelism

- ▶ The potential for parallelism exists at three levels in a contemporary multicore system:
 - ▶ Hardware parallelism within each core processor, known as instruction level parallelism,
 - ▶ Multiprogramming and multithreaded execution within each processor,
- ❑ In multiprogramming, the I/O burst of one process overlaps with the CPU/ burst of another.
- ❑ In multithreading, each processor overlaps different threads the same way processes are overlapped in multiprogramming.

Potential for parallelism

- ▶ The potential for parallelism exists at three levels in a contemporary multicore system:
 - ▶ Hardware parallelism within each core processor, known as instruction level parallelism,
 - ▶ Multiprogramming and multithreaded execution within each processor,
 - ▶ Potential for a single application to execute in concurrent processes or threads across multiple cores.
- ❑ An application contains multiple processes (or threads).
- ❑ Multiple cores execute processes (or threads) of the same application.

Potential for parallelism

- ▶ The potential for parallelism exists at three levels in a contemporary multicore system:
 - ▶ Hardware parallelism within each core processor, known as instruction level parallelism,
 - ▶ Multiprogramming and multithreaded execution within each processor,
 - ▶ Potential for a single application to execute in concurrent processes or threads across multiple cores.
- ▶ Without strong and effective OS support, hardware resources will not be efficiently used.



Strategies for multicore OS design

- ▶ There are two approaches which system designers use to create operating systems which work well with multicore machines:
 - ▶ Techniques to exploit parallelism in applications
 - ▶ Virtual machine approach

Strategy# 1: Techniques to exploit parallelism within applications

- ▶ Most applications can be subdivided into multiple tasks that can execute in parallel.
- ▶ These tasks can be implemented as multiple processes or multiple threads.
- ▶ The difficulty is that the developer must decide how to split up the application work into independently executable tasks.
- ▶ It is primarily the compiler and the programming language features that support the parallel programming design process.
- ▶ OS can support this design process by efficiently allocating resources among parallel tasks as defined by the developer.



Example:

Grand Central Dispatch of Mac OS X



- ▶ Mac OS X 10.6 includes a multicore support capability known as Grand Central Dispatch (GCD).
- ▶ GCD is a thread pool mechanism, in which the OS maps tasks onto threads.
- ▶ It is a new and valuable tool for exploiting the available parallelism of a multicore system.
- ▶ Developers are encouraged to identify functions of their applications that would be better executed in another thread than the main thread.
- ▶ GCD makes it easy to break off the entire unit of work while maintaining the existing order and dependencies between subtasks.

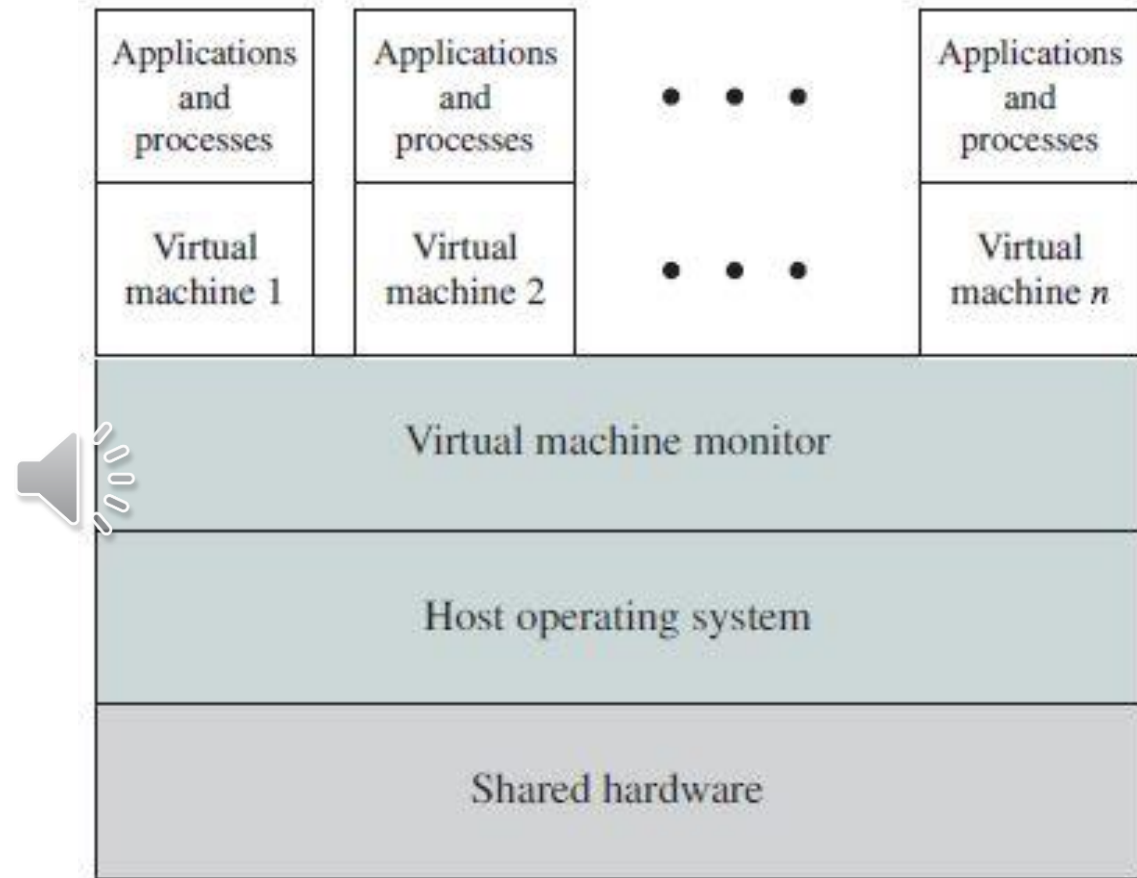
Virtualization

virtual machines

- ▶ Virtualization technology enables a single PC or server to simultaneously run multiple operating systems or multiple sessions of a single OS.
- ▶ A machine with virtualization can host numerous applications, including those that run on different operating systems, on a single platform.
- ▶ The host operating system can support several virtual machines (VM), each of which has the characteristics of a particular OS and, in some versions of virtualization, the characteristics of a particular hardware platform.

Virtual machines

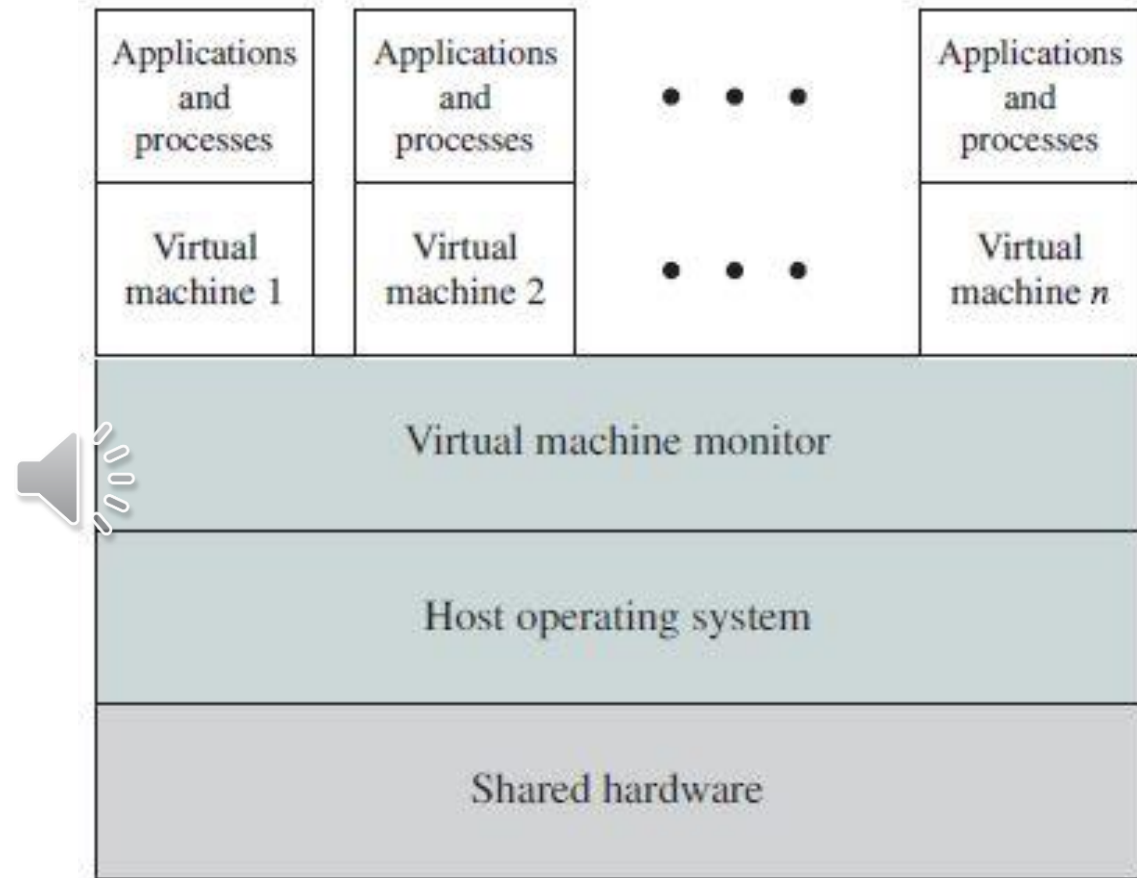
- ▶ The virtual machine monitor (VMM), or hypervisor, runs on top of the host OS.
- ▶ The VMM supports virtual machines (VMs), which are emulated hardware devices.
- ▶ Each VM runs a separate OS.
- ▶ The VMM handles each operating system's communications with the processor, the storage medium, and the network.



Virtual machines

► To execute programs, the VMM hands off the processor control to a virtual OS on a VM.

► Most VMs use virtualized network connections to communicate with one another, when such communication is needed.

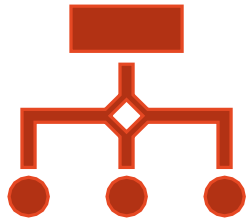




Strategy# 2: Virtual Machine Approach

- ▶ We allow one or more cores to be dedicated to a process and then leave the processor alone to devote its efforts to that process.
- ▶ In this way, we avoid much of the overhead of task switching and scheduling decisions.
- ▶ The multicore OS could then act as a hypervisor that makes a high-level decision to allocate cores to applications.
- ▶ When we interleave processes, like we do in time-sharing, switching overhead grows and starts affecting the machine's response times.
- ▶ With many-core systems, this interleaving and switching becomes unnecessary, as we are not functioning with one processor anymore.
- ▶ The OS assigns an application a processor and some memory, and the program itself, using metadata generated by the compiler, would best know how to use these resources.

Comparison of strategies for multicore OS design

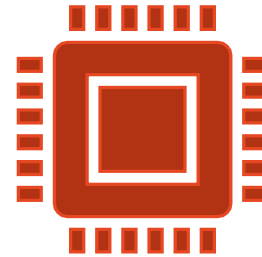


Techniques to exploit parallelism in applications

Identify parts of a process that would run in parallel,

Map these parts to different executable entities, and

Execute them on different cores



Virtual machine approach

Assign each executable entity to a separate core, and

Let the executable entity start and end at the same core without interruption.

Multicore and Multithreading



Performance issues and benefits

Performance of software on multicores

- ▶ The potential performance benefits of a multicore organization depend on the ability to effectively exploit the parallel resources available to the application.

- ▶ Amdahl's law states that:



$$\text{Speedup} = \frac{\text{program execution time on a uniprocessor}}{\text{program execution time on } N \text{ parallel processors}}$$

$$\text{Speedup} = 1 / \left(1 - f + \frac{f}{N} \right)$$

- ▶ The law assumes a program in which a fraction $(1 - f)$ of the execution time involves code that is inherently serial, and a fraction f of the execution time involves code that is infinitely parallelizable with no scheduling overhead.

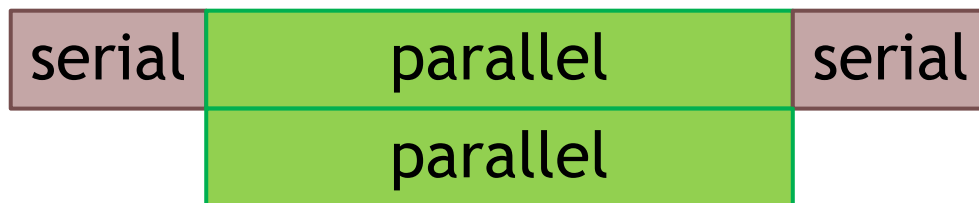


On 1 processor

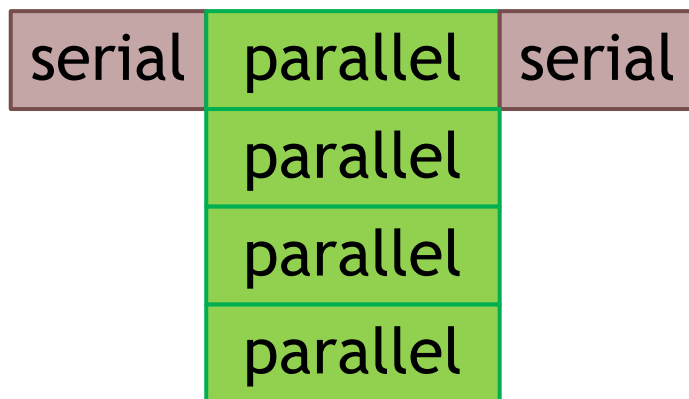


Amdahl's Law

On 2 processors



On 4 processors



- Assume that 90% of a program can be parallelized.

$$f = 0.9$$

- A system has 10 cores.

$$N = 10$$

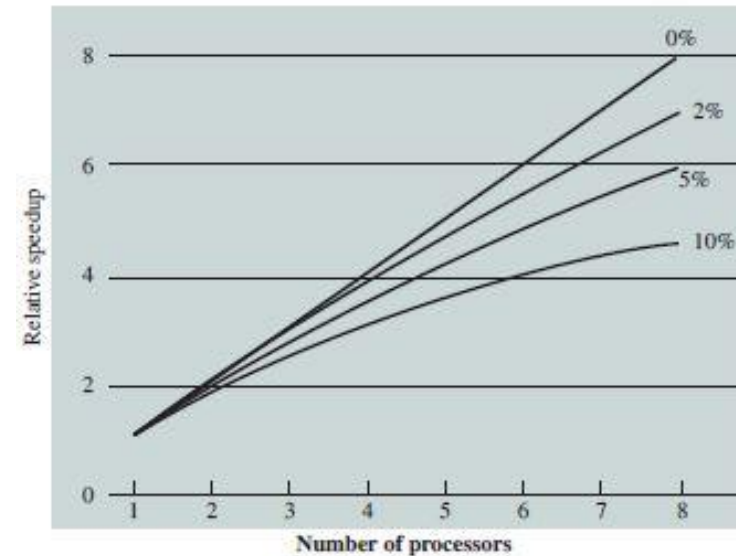
$$S = 1 / (1 - 0.9 + 0.9 / 10)$$

$$S = 5.26$$

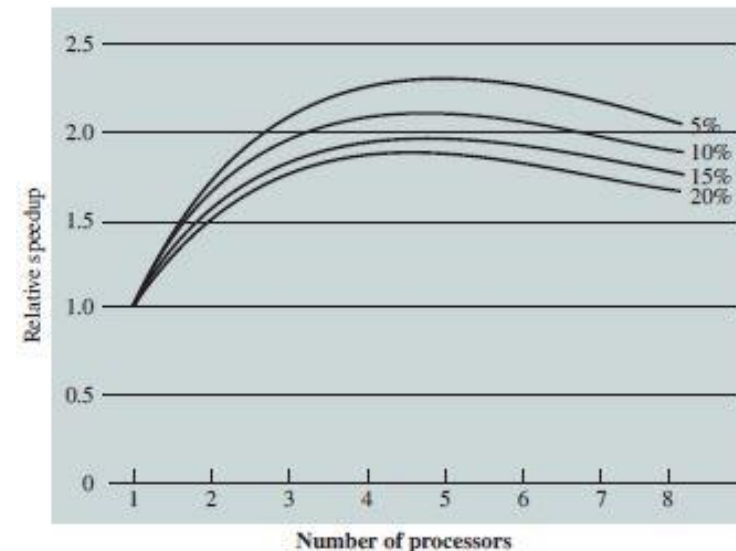
- If I invest in a machine containing 10 cores, I will get only 5 times the speed I achieved on a uniprocessor!

Expected versus actual performance

- ▶ In reality, software typically incurs overhead as a result of communication and distribution of work to multiple processors.
- ▶ This results in a curve where performance peaks and then begins to degrade because of the increased burden of the overhead of using multiple processors.



(a) Speedup with 0%, 2%, 5%, and 10% sequential portions



(b) Speedup with overheads

Figure 4.7 Performance Effect of Multiple Cores

Using multicores for databases

- ▶ In a set of database applications, the serial fraction was reduced within hardware architectures, operating systems, middleware, and the database application software.
- ▶ Multicore systems can be used effectively in database management systems and database applications, and in many kinds of servers, because servers handle numerous relatively independent transactions in parallel.

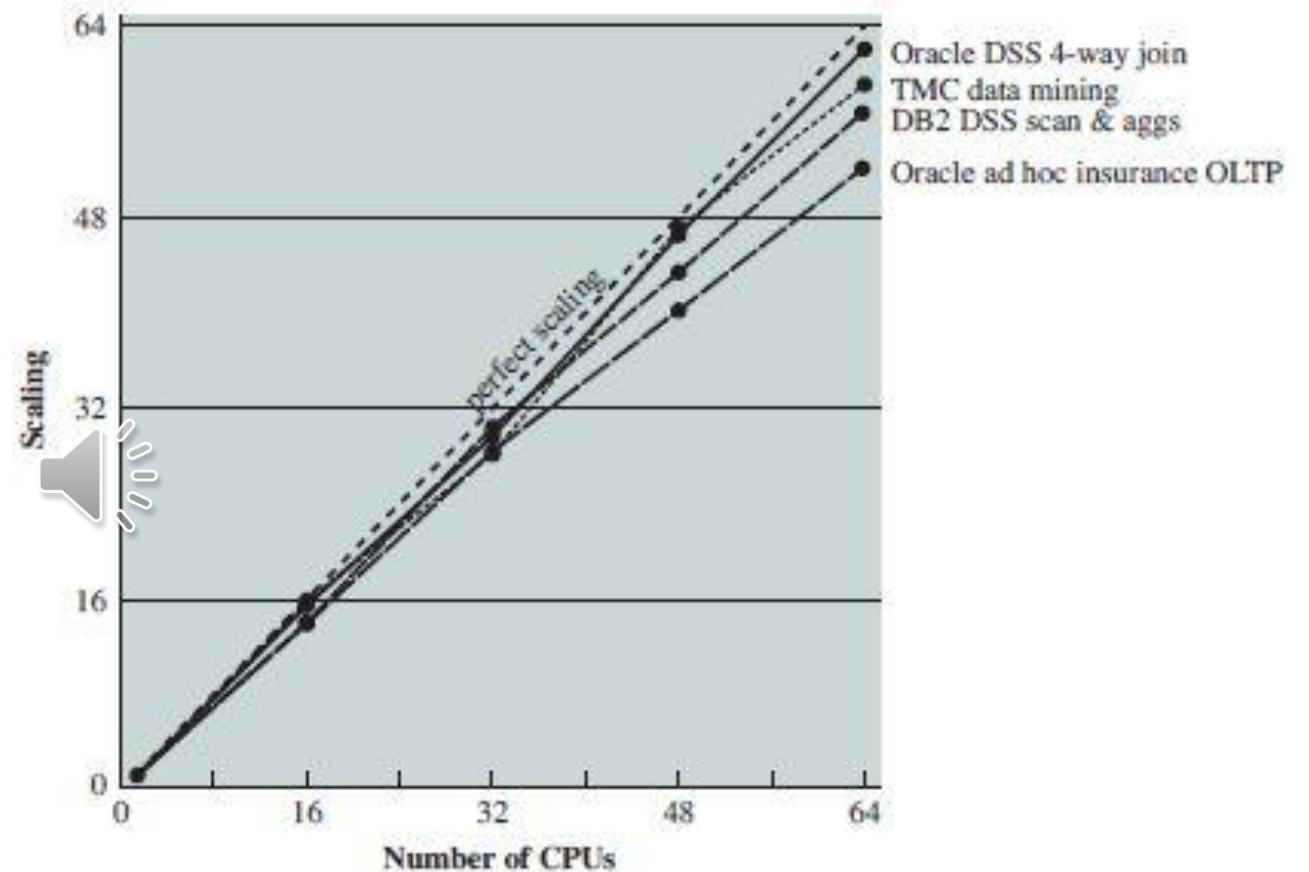


Figure 4.8 Scaling of Database Workloads on Multiple-Processor Hardware

Applications with the ability to scale throughput with the number of cores



- ▶ Multithreaded native applications
 - ▶ Have a small number of highly threaded processes.
 - ▶ Examples: Lotus Domino or Siebel CRM (Customer Relationship Manager)
- ▶ Multiprocess applications
 - ▶ Have many single-threaded processes.
 - ▶ Examples: Oracle database, SAP, and PeopleSoft
- ▶ Multi-instance applications
 - ▶ Run multiple instances of the application in parallel.
- ▶ Java applications
 - ▶ Java language facilitates multithreaded applications.
 - ▶ Java Virtual Machine provides scheduling and memory management for Java applications.
 - ▶ Examples: Sun's Java Application Server, BEA's Weblogic, IBM's Websphere, and the open-source Tomcat application server, all applications that use a Java 2 Platform, Enterprise Edition (J2EE platform) application server



Homework:

After this lecture you should be better aware of the computing resources you use daily. Find out the following:

1. Is your laptop a uniprocessor, a multiprocessor or a multicore ? What is the name of the processor? Give some of its specifications.
2. Is your smartphone a uniprocessor, a multiprocessor or a multicore ? What is the name of the processor? Give some of its specifications.
3. What kind of threads does the OS on your laptop support?

Lessons learned

- ▶ We explored multicore architecture.
- ▶ We realized that multicores and multithreading strengthen each other. 
- ▶ We discussed the design issues of an operating system for multicores.
- ▶ We listed the applications that show improved performance when working over multicores.

Things to do



Read the book.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.